

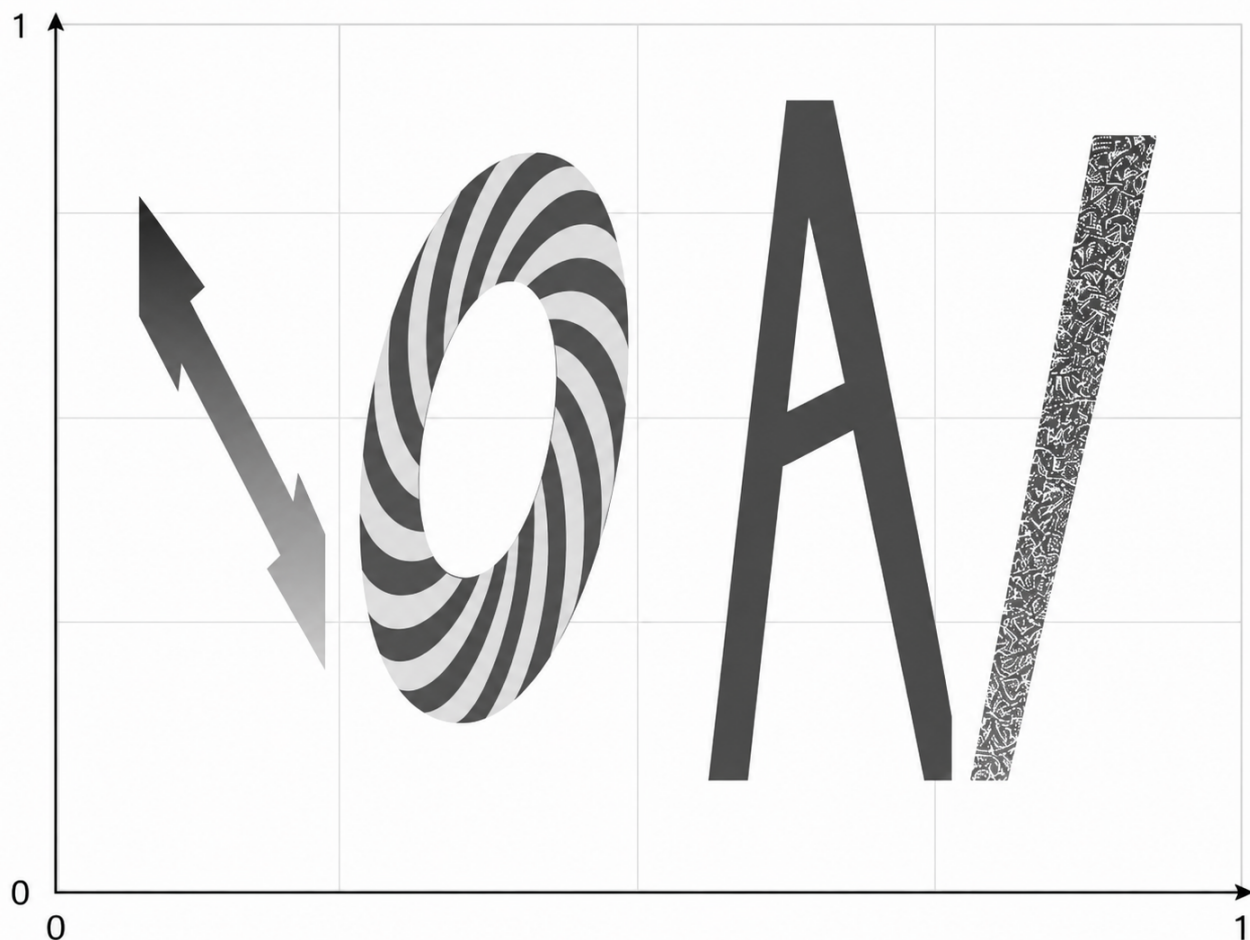
Поле IOAI

- **Ограничение по времени:** 5 minutes
- **Хранилище:** 5 GB
- **Размер решения:** `solution.ipynb`, `custom_model.py` ≤ 1 MB суммарно
- **Предобученные модели:** отсутствуют — обучение с нуля, без доступа к интернету во время проверки
- **Бейзлайн:** 31.2187

Задача

Мэр Астаны хочет украсить город стилизованными логотипами IOAI. Как статистик, он рассматривает всё, включая логотип, как пространственную функцию $F(x, y, \overline{W})$, где $x, y \in [0, 1]$ представляют собой координаты на 2D-плоскости, а $\overline{W}$ — набор скрытых параметров, определяющих такие стилистические характеристики, как цвета и углы наклона букв.

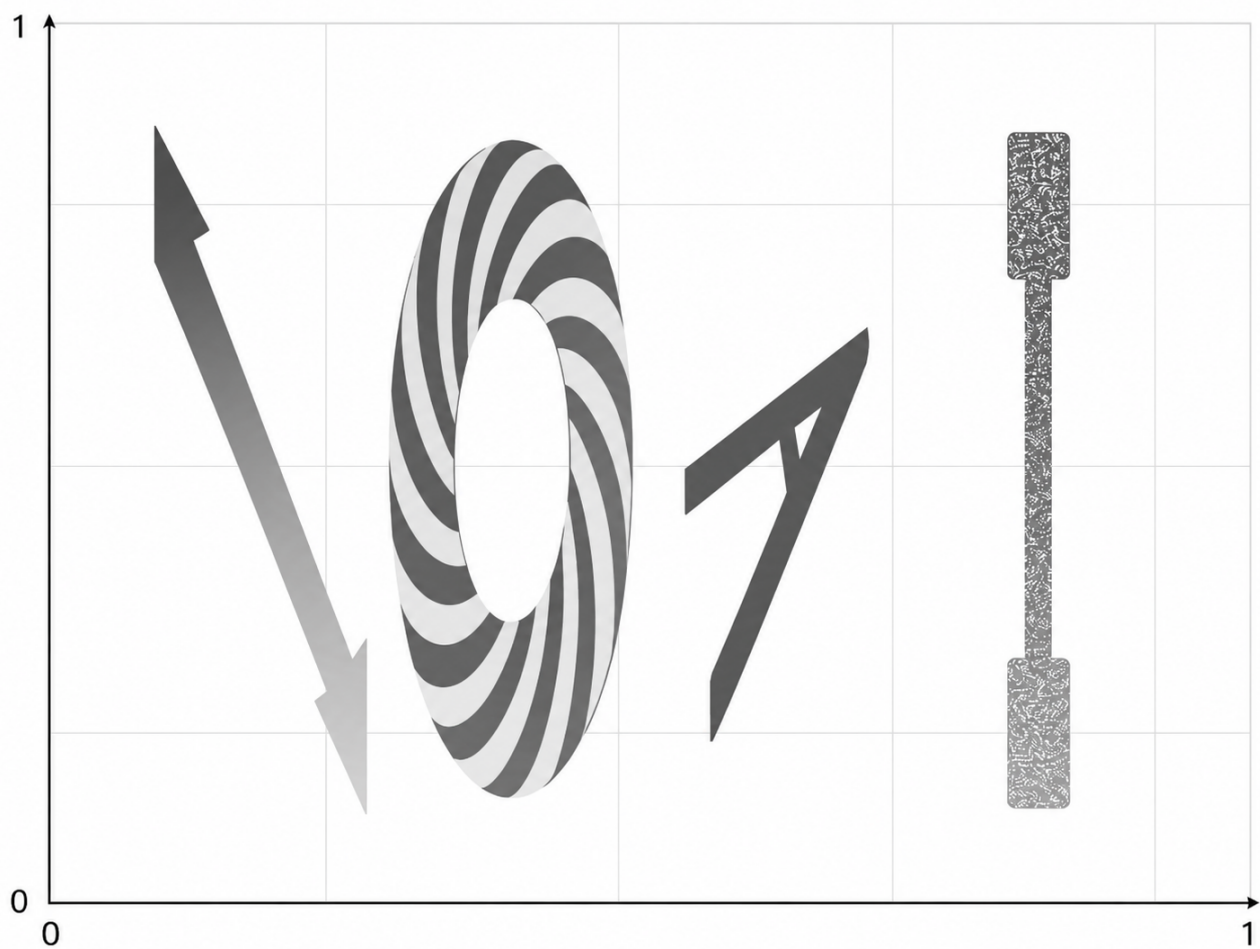
Поскольку F слишком сложна, чтобы выразить её явным математическим уравнением, ваша задача — обучить нейронную сеть аппроксимировать её. Для любой пары координат (x, y) сеть будет выдавать значение **поля IOAI**, формируя полную визуализацию логотипа в виде тепловой карты на всей плоскости. Ниже приведён пример визуализации F в виде тепловой карты при некоторых конкретных скрытых параметрах $\overline{W}$.

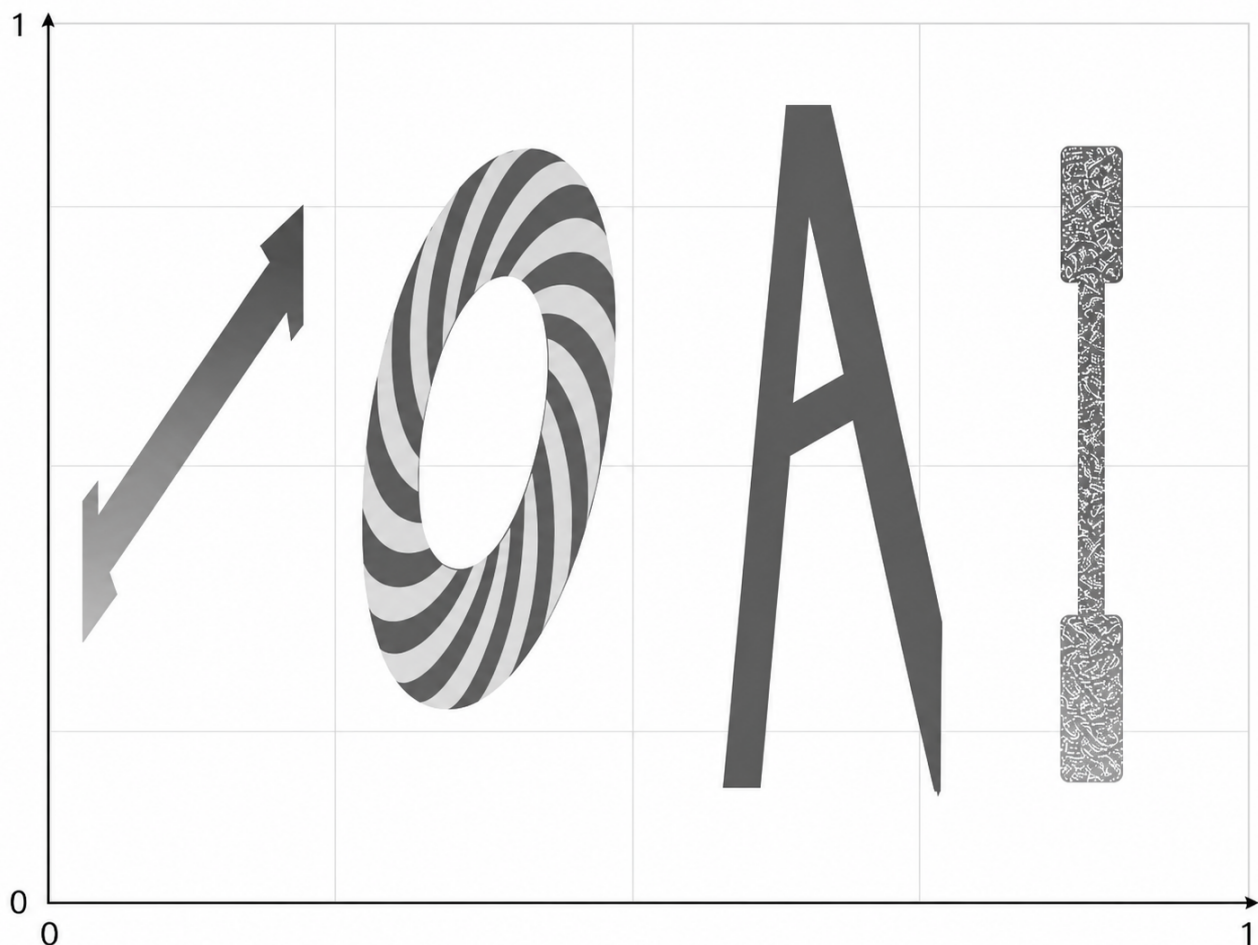


Из чего состоит поле IOAI? Из четырёх букв и фона.

- Значения внутри первой буквы I очень велики ($1e+10$ и более) и имеют линейный градиент
- Значения в букве O образуют спиральный узор
- Значение внутри буквы A всегда равно -1
- Значения внутри последней буквы I должны быть случайными значениями из диапазона $[-2026, 2026]$, даже если вычисление дважды выполняется в одной и той же точке
- Вне букв значение всегда равно нулю

Функция имеет скрытые параметры $\overline{W}$, которые влияют на масштаб и наклон букв, а также на диапазон значений внутри первой буквы I . Однако буквы не будут пересекаться. Ниже приведено несколько иллюстративных примеров того, как выглядит поле IOAI при различных $\overline{W}$:





Что вам дано:

Эта задача НЕ содержит датасетов. Вместо этого вам предоставляется функция-генератор, которая настраивается конфигом JSON из файла `data/train_config/field_config.json`.

Тестовый конфиг скрыт, но имеет схожий характер. Ваша задача — обучиться на предоставленном генераторе, используя столько данных, сколько пожелаете. Ваши «обучающее» и «тестовое» распределения генерируются одним и тем же генератором — вы лишь не знаете, в каких точках (x_i, y_i) будет проводиться оценивание.

Ваша посылка должна состоять из:

- класса обучаемой модели, сохранённого как `custom_model.py`. Эта модель должна наследоваться от класса `torch.nn.Module` и использовать только импорты `torch`. Она должна содержать класс `CustomModel`, используемый в ноутбуке `solution.ipynb`.
- ноутбука `solution.ipynb`, который создаст веса `model.pt`

Оценивание

Для каждой области минимальная оценка равна 0, а максимальная — 1. Итоговая оценка усредняется по всем пяти областям (по одной для каждой из четырёх букв и для фона) и

умножается на 100. Предусмотрен **штраф за количество параметров**:

Если в вашей модели более 20260 параметров, оценка уменьшается вдвое.

Количество параметров измеряется с помощью `sum(p.numel() for p in model.parameters())`. Мы ожидаем, что ваша модель также будет работать в стохастическом режиме, причём `nn.Dropout` из PyTorch должен быть частью модели.

Для стандартных областей

Для каждой области R (первая буква I, O, A, Background) мы оцениваем модель на $N_R = 512$ тестовых точках (x_i, y_i) с истинными значениями v_i и предсказаниями $\hat{v}_i$. В качестве основной метрики мы используем нормализованную среднюю абсолютную ошибку (MAE). MAE определяется следующим образом:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Нормализация выполняется следующим образом:

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

где $s_R > 0$ — константа масштаба.

Для области последней буквы I

В этой области **дропаут включён во время оценивания**. Для каждой тестовой точки j :

1. Мы запускаем модель $K = 10$ раз, чтобы получить $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Если какой-либо выход находится вне диапазона $[-2026, 2026]$, то $\text{pointScore}(j) = 0$.
3. В противном случае вычисляется стандартное отклонение σ_j для K выходов и преобразуется в оценку:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

где $s_E > 0$ — фиксированная константа масштаба.

Оценка области представляет собой среднее значение по всем точкам этой области:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

где $N_E = K * N_R$.

Проще говоря, чем больше разнообразие, тем выше будет ваша оценка за эту область. **Вы не можете использовать случайность в чистом виде, включая функции PyTorch `rand*` и `_uniform`; случайность должна возникать при инференсе с включённым дропаутом.**

Как отправить решение

1. Откройте `solution.ipynb` и выполните все ячейки.
2. Улучшите модель `CustomModel` в `custom_model.py`
3. Убедитесь, что последняя ячейка сохраняет вашу модель в файл `model.pt`.
4. На вкладке `Git` в `JupyterLab` закоммитьте `solution.ipynb` и `custom_model.py` (с комментарием), а затем отправьте его в удалённый репозиторий.
5. Вернитесь на страницу соревнования и нажмите **Отправить**. Комментарий к посылке должен совпадать с комментарием коммита из предыдущего шага.